

Knowledge Representation Methods

Representing Knowledge so that Computers can use it!

1 What is Knowledge Representation ?

- Knowledge Representation (KR) is a way to represent real-world knowledge in a form that a computer can understand, store and reason with.
- It provides a structured format to represent facts, rules, objects, relationships, etc.

"KR is the bridge between real world and the computer's reasoning."

2 Why is it Needed ?

- Real-world knowledge is complex and uncertain.
- Helps in **reasoning**, problem solving and decision making.
- Makes knowledge reusable and **maintainable**.
- Used in expert systems, robotics, NLP, etc.

3 Knowledge Representation Methods

Different methods are used depending on the type of knowledge and the application.

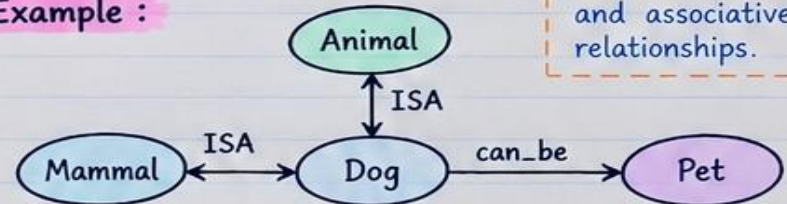
- Semantic Networks
- Frames
- Logic (Propositional & First-order)
- Production Rules
- Scripts
- Ontologies
- Probabilistic Methods

No single method is best for all situations.
Choice depends on the problem domain.

4 ① Semantic Networks

- Represents knowledge as a graph with nodes and labeled links.
- Nodes represent objects or concepts.
- Links represent relationships.

Example :



Shows hierarchical and associative relationships.

5 ② Frames

- Represents knowledge using structured data structures called frames.
- A frame has slots (**attributes**) and **values**.
- Useful for representing stereotyped situations (objects, events).

Example :

Frame : Car
Slots :
- brand : Toyota
- color : Red
- no_of_wheels : 4
- type : Sedan

Frame representation of a car object.

6 ③ Logic (Propositional & First-Order Logic)

- Represents knowledge using logical statements (facts, predicates, quantifiers).
- Allows formal reasoning (inference).
- Propositional Logic** : deals with simple true/false statements.
- First-Order Logic (FOL)** : represents objects, properties and relationships using predicates.

Example (FOL) :

$\forall x (Human(x) \rightarrow Mortal(x))$
Human(Socrates)
 $\therefore Mortal(Socrates)$

If x is a human then x is mortal.
Since Socrates is a human, therefore Socrates is mortal.

7 ④ Production Rules

- Represents knowledge as IF-THEN rules.
- Widely used in expert systems.

Example :

IF Temperature > 100
THEN Boil = True

If temperature is greater than 100, then water boils.

8 ⑤ Scripts

- Represents knowledge about common sequences of events (stereotyped situations).
- Useful for understanding everyday activities.

Example :

Script : Restaurant
1. Enter restaurant
2. Get seated
3. Order food
4. Eat food
5. Pay bill
6. Leave

Helps in understanding typical event sequences.

9 ⑥ Ontologies

- A formal and explicit specification of shared concepts in a domain.
- Defines classes, relationships, constraints.
- Used in Semantic Web, knowledge sharing, AI.

Example :

Class : Student
↳ hasProperty : name, rollNo
↳ subclassOf : Person

10 ⑦ Probabilistic Methods

- Represents uncertain or incomplete knowledge using probabilities.
- Commonly uses Bayesian Networks.

Example :

$P(Rain) = 0.3$
 $P(WetGround | Rain) = 0.8$

Helps in reasoning under uncertainty.

"Knowledge representation is the foundation of intelligent behavior."

Represent • Reason • Solve • Apply !

Mappings

"A mapping shows how elements of one set are related to elements of another set."

Definition

A **mapping** (or function) is a relation from a set A (called **domain**) to a set B (called **codomain**) that assigns each element of A to **exactly one** element of B .

Notation: $f : A \rightarrow B$

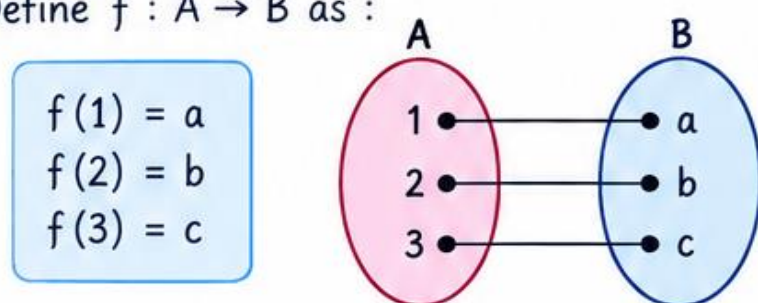
(read as "f maps A to B")

Terminology

- A : **Domain** (set of input elements)
- B : **Codomain** (set of possible output elements)
- $f(a)$: **Image** of $a \in A$ under f
- **Range** : Set of actual output elements $\{f(a) \mid a \in A\}$
- **Pre-image** : For $b \in B$, the set $\{a \in A \mid f(a) = b\}$

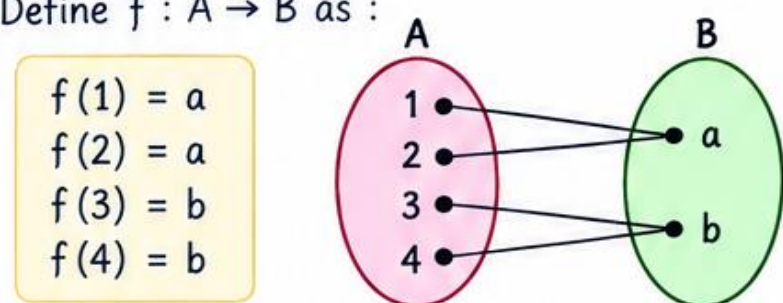
Example 1 : Simple Mapping

Let $A = \{1, 2, 3\}$ and $B = \{a, b, c\}$.
Define $f : A \rightarrow B$ as :



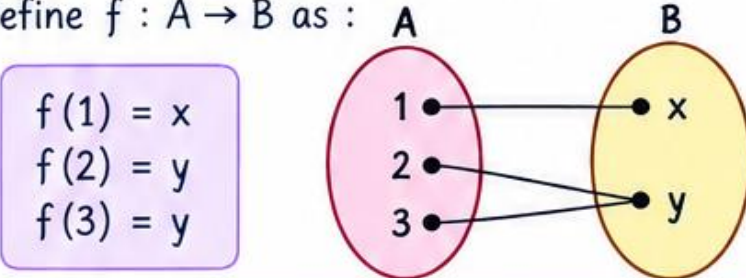
Example 2 : Many-to-One Mapping

Let $A = \{1, 2, 3, 4\}$ and $B = \{a, b\}$.
Define $f : A \rightarrow B$ as :



Example 3 : Onto (Surjective) Mapping

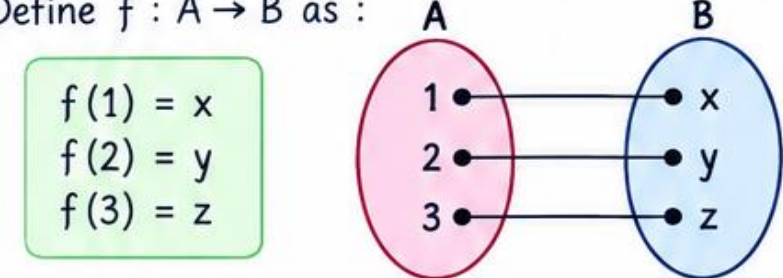
Let $A = \{1, 2, 3\}$ and $B = \{x, y\}$.
Define $f : A \rightarrow B$ as :



Every element in B has at least one pre-image.
 $\therefore f$ is onto (surjective).

Example 4 : One-to-One (Injective) Mapping

Let $A = \{1, 2, 3\}$ and $B = \{x, y, z\}$.
Define $f : A \rightarrow B$ as :



Different elements in A map to different elements in B .
 $\therefore f$ is one-to-one (injective).

Types of Mappings

Type	Property	Example
One-to-One (Injective)	Distinct elements in A have distinct images in B .	$f(1) = x$ $f(2) = y$ $f(3) = z$
Many-to-One	Two or more elements in A map to the same element in B .	$f(1) = a$ $f(2) = a$ $f(3) = b$
Onto (Surjective)	Every element in B has at least one pre-image in A .	$f(1) = x$ $f(2) = y$ $f(3) = y$
One-to-One and Onto (Bijective)	Both injective and surjective (perfect matching).	$f(1) = a$ $f(2) = b$ $f(3) = c$

Key Points

- Every element of the domain must have exactly one image in the codomain.
- $\text{Range} \subseteq \text{Codomain}$.
- A function can be one-to-one, onto, both (bijective), or neither.
- Mappings are widely used in mathematics, computer science and AI for representing relationships between objects.

Applications

- Knowledge representation (mapping real-world entities to symbols).
- Database systems (mapping keys to values).
- Programming (function inputs to outputs).
- State-space representation in AI (mapping states to successors).
- Graph theory (mapping nodes or edges).

"Mappings help us represent knowledge in a structured and unambiguous way."

Instance and ISA Relationships

"ISA → is-a
Instance → is an example of"

1 Introduction

- In knowledge representation, we represent real-world knowledge using objects, classes and their relationships.
- Two important relationships are :
 - Instance relationship** (instance-of)
 - ISA relationship** (is-a)

These relationships help in organizing knowledge in a hierarchical manner and support inheritance of properties.

2 Instance Relationship

- An instance is a **specific individual object** that belongs to a class.
- Notation** : `instance(object, Class)`
- Read as : "object is an instance of Class".
- It connects an individual to a concept (class).

Example :

```
instance(Tom, Person)
instance(Rex, Dog)
instance(Delhi, City)
```

Tom is an instance of the class Person.

3 ISA Relationship

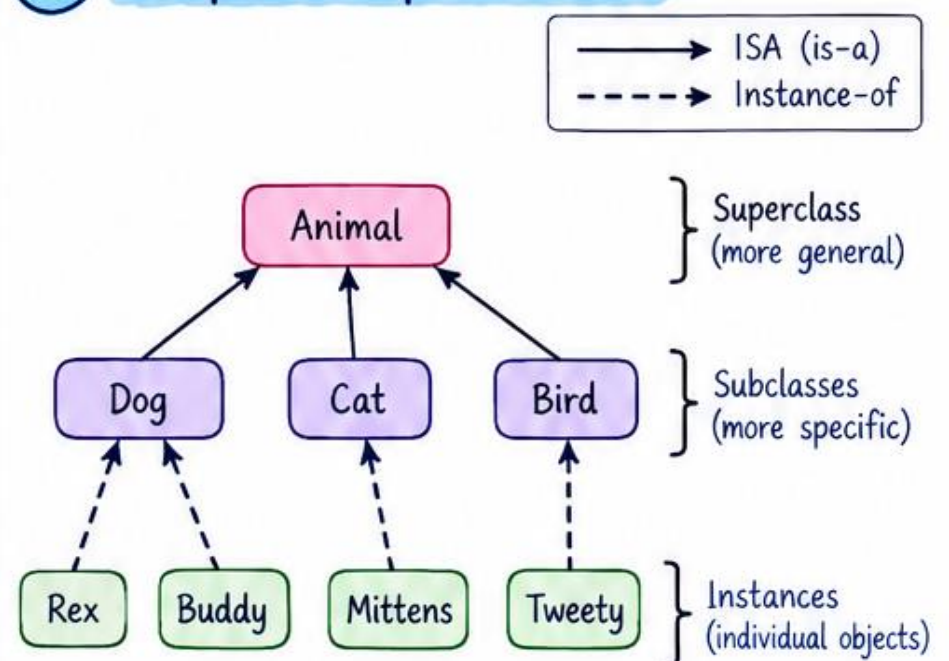
- ISA (is-a) represents a subclass-superclass relationship between two classes.
- It is used to denote that one class is a specialized form of another class.
- Notation** : `ISA(Subclass, Superclass)`
- Read as : "Subclass is a kind of Superclass".
- Properties of a superclass are inherited by its subclasses.

Example :

```
ISA(Dog, Animal)
ISA(Cat, Animal)
ISA(Car, Vehicle)
ISA(Sedan, Car)
```

A Dog is a kind of Animal.

4 Graphical Representation



5 More Examples

1 Person Hierarchy

```
ISA(Student, Person)
ISA(Teacher, Person)
instance(Alice, Student)
instance(Bob, Teacher)
```

Alice is a student, so she is also a person (inherits properties of Person).

2 Vehicle Hierarchy

```
ISA(Car, Vehicle)
ISA(Truck, Vehicle)
instance(HondaCity, Car)
instance(TataTruck, Truck)
```

HondaCity is a car, so it is also a vehicle.

3 Organization Hierarchy

```
ISA(University, Organization)
instance(GTU, University)
```

GTU is an instance of University, and hence also an Organization.

6 Key Points

- Instance** links an individual object to a class.
- ISA** links a subclass to a more general superclass.
- Subclasses **inherit properties** from superclasses.
- Used to build hierarchical knowledge representations (e.g., semantic networks, ontologies).
- Supports reasoning (e.g., if Rex is a Dog and Dog is an Animal, then Rex is an Animal).

7 Applications

- Semantic networks and ontologies
- Object-oriented knowledge representation
- Natural language understanding
- Expert systems
- Inheritance of properties and reasoning

"Instances give us concrete examples, ISA gives us generalization."

Forward Reasoning

"Start from what is known and infer new knowledge."

1 Definition

- Forward reasoning (also called **data-driven reasoning**) is an inference method that starts from **known facts** and repeatedly applies **inference rules** to **derive new facts** until the goal is reached or no more facts can be inferred.
- It is a **bottom-up approach** in reasoning.

2 Basic Idea

- Start with the initial set of **known facts**.
- Find rules whose **conditions** are satisfied by the current facts.
- Fire (apply)** those rules.
- Add the new facts to the knowledge base.
- Repeat the process until the goal is derived or no new facts can be inferred.

3 Rule Representation

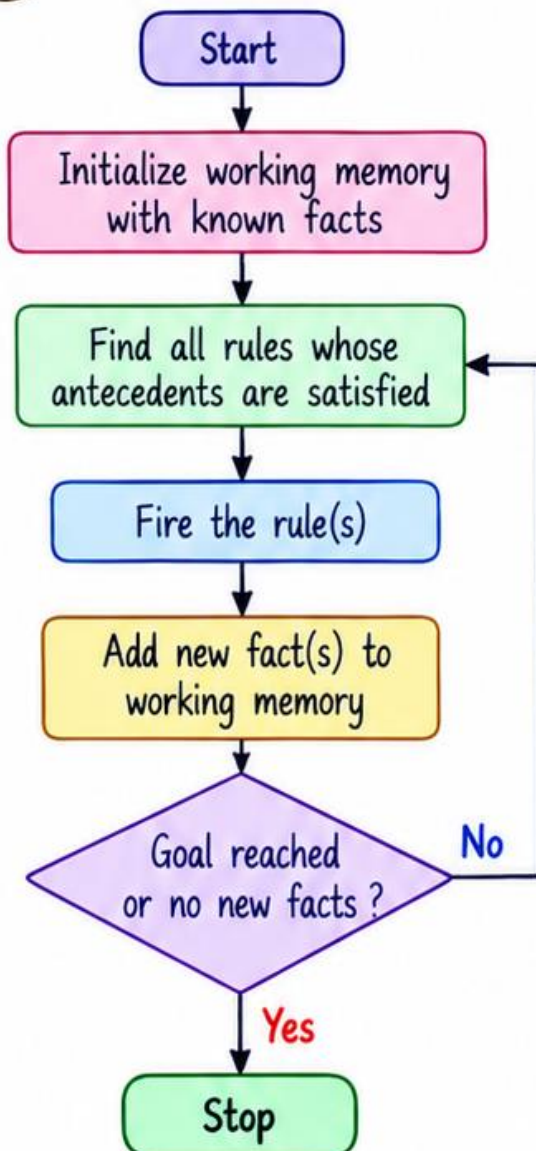
- Rules are written as **IF - THEN** statements.

IF condition(s)
THEN conclusion(s)

Example :

IF $A \wedge B$
THEN C

4 Algorithm



5 Example : Simple Inference

Knowledge Base

Facts :

- A
- B

Rules :

- R1 : IF A THEN C
R2 : IF B THEN C
R3 : IF C THEN D

Step-by-Step Execution

- Initial facts = {A, B}
- Apply R1 ($A \rightarrow C$) $\Rightarrow$ {A, B, C}
- Apply R3 ($C \rightarrow D$) $\Rightarrow$ {A, B, C, D}
- Goal D is reached ✓

Example : Medical Diagnosis (Brief)

Rules :

- R1 : IF Fever $\wedge$ Cough THEN Flu
R2 : IF Flu THEN Viral_Infection
R3 : IF Viral_Infection THEN Rest

Initial Facts : Fever, Cough

Goal : Rest

Inference Steps :

- Facts = {Fever, Cough}
- Apply R1 $\Rightarrow$ Flu
Facts = {Fever, Cough, Flu}
- Apply R2 $\Rightarrow$ Viral_Infection
Facts = {Fever, Cough, Flu, Viral_Infection}
- Apply R3 $\Rightarrow$ Rest
Goal Rest is reached ✓

6 Advantages

- ✓ Simple and intuitive.
- ✓ Good for problems with large amount of data.
- ✓ Can derive all possible conclusions.
- ✓ Widely used in expert systems.

7 Applications

- Expert systems (e.g., medical diagnosis)
- Decision support systems
- Configuration systems
- Natural language understanding
- Planning and scheduling
- Fault diagnosis

"From facts to knowledge – step by step."

Backward Reasoning

Artificial Intelligence
(7th Sem - GTU)

1) Introduction :

- Backward reasoning is a **goal-driven** reasoning technique.
- It starts with a query (goal) and works **backwards** to find the facts that can prove the goal.
- It uses rules to reduce the goal into sub-goals until all sub-goals are satisfied by known facts in the knowledge base.
- If all sub-goals are proved, the goal is true. If no rules can prove the goal, it fails.

2) Key Idea :

- Start with the goal (query).
- Find rules whose conclusion matches the goal.
- Replace the goal with the rule's antecedents (sub-goals).
- Repeat recursively until you reach known facts.

Backward reasoning is used in logic programming (e.g. Prolog) where the system tries to prove a query using a set of facts and rules.

3) Backward Reasoning vs Forward Reasoning.

Aspect	Backward Reasoning	Forward Reasoning
Strategy	Goal-driven	Data-driven
Start from	Goal (query)	Known facts
Direction	Works backwards	Works forwards
Used in	Prolog (logic programming)	Expert systems
More efficient when	Specific goal is given	Need to derive all possible conclusions

4) Process (General Steps)



5) Example (AI Domain) :

Knowledge Base (Rules & Facts) :

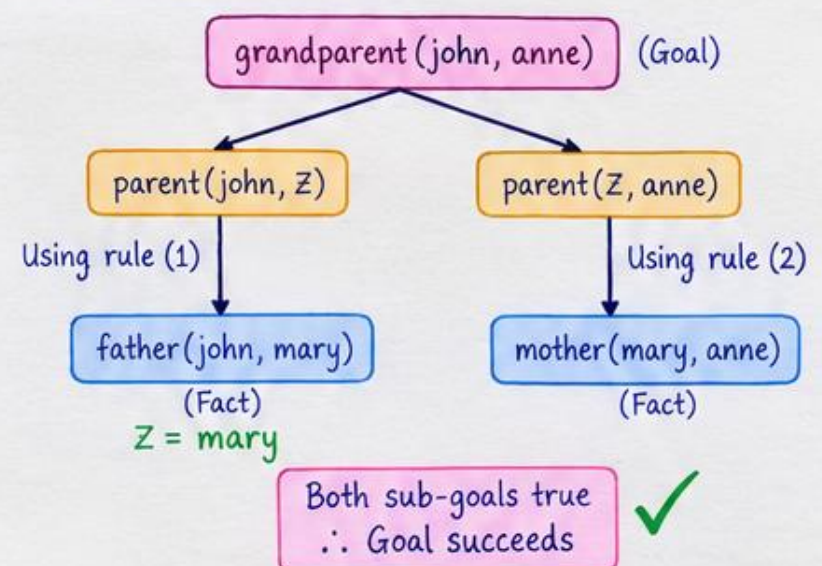
1. $\text{parent}(X, Y) :- \text{father}(X, Y).$
2. $\text{parent}(X, Y) :- \text{mother}(X, Y).$
3. $\text{grandparent}(X, Y) :- \text{parent}(X, Z), \text{parent}(Z, Y).$
4. $\text{father}(\text{john}, \text{mary}).$
5. $\text{mother}(\text{mary}, \text{anne}).$

Query : $?- \text{grandparent}(\text{john}, \text{anne}).$

Explanation :

- ① To prove $\text{grandparent}(\text{john}, \text{anne})$, we use rule (3).
- ② It requires two sub-goals : $\text{parent}(\text{john}, Z)$ and $\text{parent}(Z, \text{anne})$.
- ③ $\text{parent}(\text{john}, Z)$ is proved using rule (1) with fact $\text{father}(\text{john}, \text{mary}) \Rightarrow Z = \text{mary}$.
- ④ $\text{parent}(\text{mary}, \text{anne})$ is proved using rule (2) with fact $\text{mother}(\text{mary}, \text{anne})$.
- ⑤ Since both sub-goals are true, the query succeeds.
 $\therefore \text{john is a grandparent of anne.}$

6) Derivation Tree :



7) Prolog Program :

```

parent(X, Y) :- father(X, Y).
parent(X, Y) :- mother(X, Y).
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).

father(john, mary).
mother(mary, anne).

?- grandparent(john, anne).
  
```

Output :
true. ✓

8) Applications :

- Expert systems
- Diagnostic systems (e.g. medical diagnosis)
- Query answering in knowledge bases
- Theorem proving

9) Advantages :

- ✓ Efficient when a specific goal is given.
- ✓ Avoids irrelevant facts/rules.
- ✓ Widely used in Prolog and logic-based systems.

10) Limitations :

- ✗ May be inefficient if many rules apply.
- ✗ Can lead to infinite loops (if not handled properly).
- ✗ Depends on the availability of suitable rules.

"Backward reasoning starts with a question and ends with an answer." 😊

Forward Reasoning - Example

AI
(7th Sem-GTU)

1) Idea :

- Start from known facts (in knowledge base).
- Apply rules to infer new facts.
- Continue forward until the goal is reached or no more facts can be inferred.
- It is a **data-driven** reasoning approach.

2) Example Problem :

Determine whether a person is a grandparent using forward reasoning.

3) Knowledge Base :

Facts :

- father(john, mary).
- mother(mary, anne).

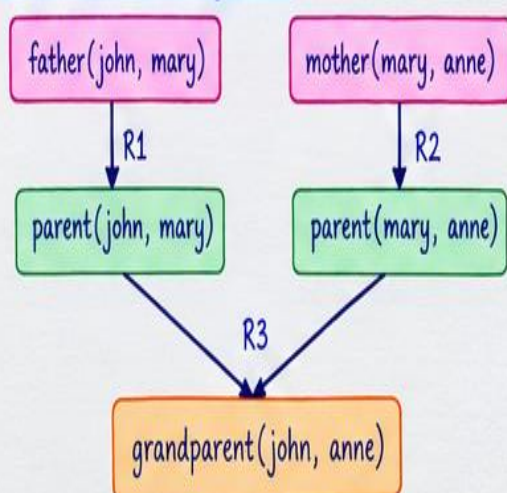
Rules :

- R1: parent(X,Y) :- father(X,Y).
 R2: parent(X,Y) :- mother(X,Y).
 R3: grandparent(X,Z) :-
 parent(X,Y), parent(Y,Z).

4) Forward Reasoning Steps :

Step	Inference	New Fact Derived	Known Facts After Step
1	From fact 1 using R1	parent(john, mary).	father(john, mary) parent(john, mary)
2	From fact 2 using R2	parent(mary, anne).	mother(mary, anne) parent(john, mary) parent(mary, anne)
3	Using R3 with parent(john, mary) and parent(mary, anne)	grandparent(john, anne).	All above facts + grandparent(john, anne)

5) Inference Diagram :



6) Result :

Using forward reasoning, we derived **grandparent(john, anne)** from the given facts.

7) Conclusion :

- Forward reasoning starts with facts and moves forward using rules.
- It is useful when we want to derive all possible conclusions (e.g. in expert systems, monitoring, diagnosis).

"Forward reasoning is driven by data." 😊

Backward Reasoning - Example

AI
(7th Sem-GTU)

1) Idea :

- Start from the goal (query).
- Look for rules whose conclusion matches the goal.
- Create sub-goals from the rule antecedents.
- Prove each sub-goal recursively until all are satisfied by known facts.
- It is a **goal-driven** reasoning approach.

2) Example Problem :

Prove the query **grandparent(john, anne)** using backward reasoning.

3) Knowledge Base :

Facts :

- father(john, mary).
- mother(mary, anne).

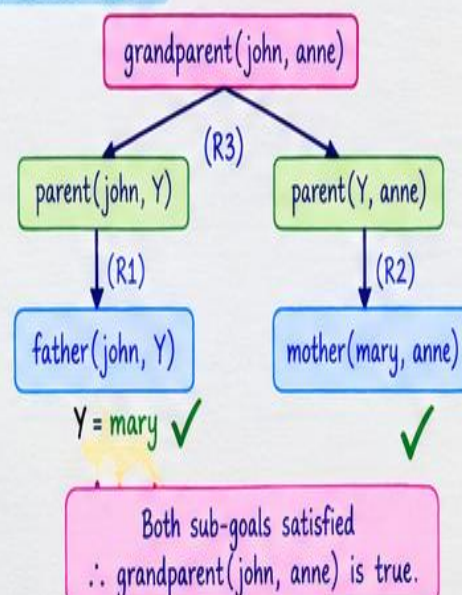
Rules :

- R1: parent(X,Y) :- father(X,Y).
 R2: parent(X,Y) :- mother(X,Y).
 R3: grandparent(X,Z) :-
 parent(X,Y), parent(Y,Z).

4) Backward Reasoning Steps :

Step	Goal (Query)	Action / Rule Used	Sub-goals	Result
1	grandparent(john, anne)	Use R3	parent(john, Y), parent(Y, anne)	New sub-goals generated
2	parent(john, Y)	Use R1	father(john, Y)	Fact matches: Y = mary
3	parent(mary, anne)	Use R2	mother(mary, anne)	Fact matches (known fact)
4	Both sub-goals true	—	—	grandparent(john, anne) proved ✓

5) Goal Tree :



6) Result :

The query **grandparent(john, anne)** is successfully proved using backward reasoning.

7) Conclusion :

- Backward reasoning starts with a goal and works backwards.
- It is useful when we want to check whether a specific query is true (e.g. in Prolog, theorem proving, query answering systems).

"Backward reasoning is driven by the goal." 😊

"Update your beliefs with evidence."

Bayesian Theorem

Artificial Intelligence
(7th Sem - GTU)

1) Introduction :

- Bayesian theorem is used to **update** the probability of a hypothesis based on new evidence.
- It is widely used in AI for uncertain reasoning, decision making, machine learning and expert systems.
- It provides a mathematical way to revise our belief given new information (evidence).

3) Terminology :

Term	Meaning
Hypothesis (H)	The event or proposition we want to verify.
Evidence (E)	Newly observed data.
Prior probability P(H)	Initial belief before seeing evidence.
Likelihood P(E H)	Probability of evidence if hypothesis is true.
Posterior probability P(H E)	Updated belief after seeing evidence.
P(E)	Total probability of evidence (can be computed using law of total probability).

6) Example : Medical Diagnosis:

A certain disease occurs in 1% of the population.
A test for the disease is 99% accurate if a person has the disease (true positive rate) and gives a false positive result 2% of the time for a person who does not have the disease. If a person tests positive, what is the probability that the person actually has the disease?

Given :

- $P(D) = 0.01$ (prior probability)
- $P(\neg D) = 0.99$
- $P(+|D) = 0.99$ (test is 99% accurate)
- $P(+|\neg D) = 0.02$ (false positive rate)

Step 1: Find P(+) using law of total probability.

$$\begin{aligned}P(+) &= P(+|D)P(D) + P(+|\neg D)P(\neg D) \\&= (0.99)(0.01) + (0.02)(0.99) \\&= 0.0099 + 0.0198 \\&= 0.0297\end{aligned}$$

Step 2: Apply Bayes' theorem.

$$\begin{aligned}P(D|+) &= \frac{P(+|D) \cdot P(D)}{P(+)} \\&= \frac{(0.99)(0.01)}{0.0297} \\&= \frac{0.0099}{0.0297} \\&\approx 0.333\end{aligned}$$

Result :

$$P(D|+) \approx 0.333'$$

Even though the test is 99% accurate, there is only 33.3% chance that the person actually has the disease!

2) Formula :

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

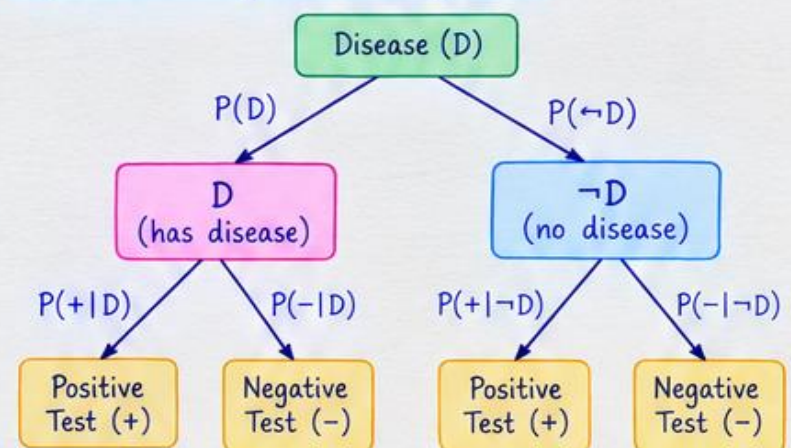
Where,

- $P(H|E)$: Posterior probability (probability of hypothesis H given evidence E)
- $P(E|H)$: Likelihood (probability of evidence E given H)
- $P(H)$: Prior probability (initial probability of H)
- $P(E)$: Evidence probability (total probability of E)

4) Intuitive Understanding :

- If evidence E is more likely to occur when H is true, then $P(H|E)$ increases.
- Helps in reasoning under uncertainty.
- Used in spam filtering, medical diagnosis, fault detection, expert systems etc.

5) Probability Tree Diagram :



7) Bayesian Theorem in Prolog :

Prolog itself does not have built-in probabilistic reasoning, but Bayesian reasoning can be implemented using probabilistic logic programming (e.g., `ProLog`, `cplint`) or by representing probabilities manually.

Example (conceptual):

```
% Prior probability
prob(disease, 0.01).
prob(no_disease, 0.99).
```

```
% Likelihoods
prob(positive_test, 0.99) :- disease.
prob(positive_test, 0.02) :- no_disease.
```

% In probabilistic Prolog (e.g., `ProLog`), queries can compute posterior probabilities.

8) Applications in AI :

- Medical diagnosis and expert systems
- Spam email detection
- Fault diagnosis in engineering systems
- Robotics (sensor fusion)
- Machine learning (Naive Bayes classifier)
- Natural language processing (text classification)

9) Advantages :

- ✓ Handles uncertainty effectively.
- ✓ Combines prior knowledge with new evidence.
- ✓ Widely used in AI and machine learning.
- ✓ Simple and elegant.

10) Limitations :

- ✗ Requires prior probabilities (sometimes hard to estimate).
- ✗ Assumes independent evidence (in many cases).
- ✗ Can be computationally expensive for complex problems.

"Bayesian reasoning turns evidence into better beliefs."



"Deals with uncertainty, just like human thinking."

Fuzzy Logic

Reasoning with Degrees of Truth

"Not everything is black or white!"

Artificial Intelligence
(7th Sem - GTU)

1) Introduction :

- Fuzzy logic is a form of multi-valued logic that deals with approximate reasoning rather than fixed and exact reasoning.
- Introduced by **Lotfi A. Zadeh** in 1965.
- Unlike classical logic (0 or 1), fuzzy logic allows partial truth (values between 0 and 1).
- Useful for real-world problems with **vagueness** and **uncertainty** (e.g., temperature is "high", not exactly 30°C).
- Widely used in AI, control systems, decision making, expert systems, and Prolog (via fuzzy extensions).

2) Classical vs Fuzzy Logic :

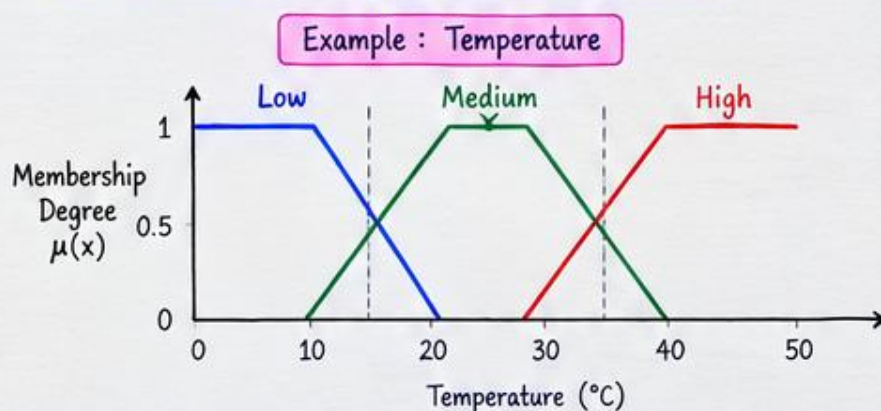
Aspect	Classical Logic	Fuzzy Logic
Truth value	Only 0 or 1	Any value between 0 and 1
Set boundaries	Crisp (sharp)	Fuzzy (gradual)
Handles uncertainty	No	Yes
Example	Age ≥ 18 (true/false)	Age is young (0.7)
Closer to	Human reasoning	Real-world reasoning

3) Fuzzy Set :

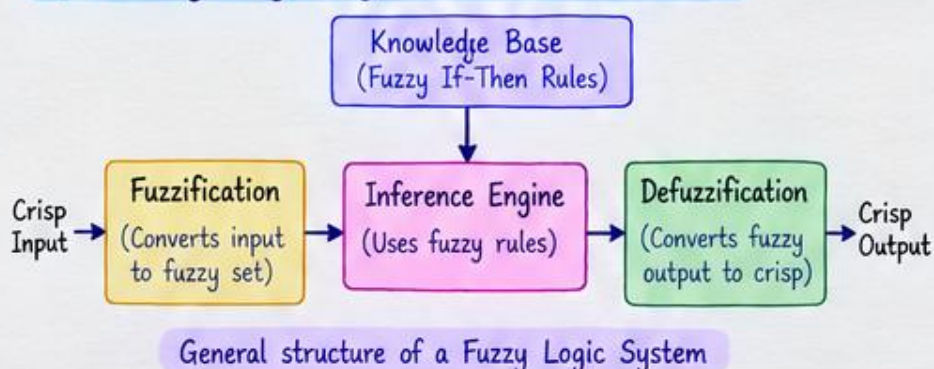
- A fuzzy set is a set with a membership function $\mu(x)$ that maps each element to a value in $[0, 1]$.
- $\mu(x)$ represents the degree of membership.
- Example: The set "High Temperature" may have $\mu(30^\circ\text{C}) = 0.5$ (partially high).

4) Membership Function :

- It defines how each input is mapped to a degree of membership (0 to 1).
- Common shapes: Triangular, Trapezoidal, Gaussian.



5) Fuzzy Logic System (Architecture) :



6) Fuzzy If-Then Rule :

- Rules are expressed in natural language.
- Format: **If <condition> Then <conclusion>**
- Example : If temperature is High Then fan speed is Fast.
- Multiple rules are used in a rule base.

7) Example : Fan Speed Control

Objective : Control the speed of a fan based on room temperature using fuzzy logic.

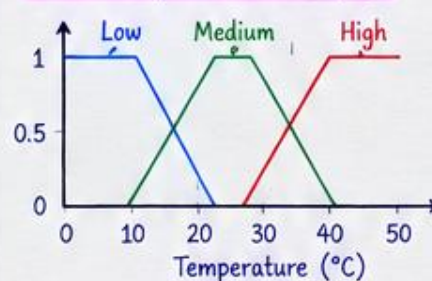
Input Variable : Temperature ($^\circ\text{C}$)

Fuzzy sets : Low, Medium, High
Range : 0 – 50 $^\circ\text{C}$

Output Variable : Fan Speed

Fuzzy sets : Slow, Medium, Fast
Range : 0 – 100 %

Membership Function (Input)



Membership Function (Output)



Fuzzy Rule Base :

- | | |
|-------------------------------|--------------------------|
| R1 : If temperature is Low | Then fan speed is Slow |
| R2 : If temperature is Medium | Then fan speed is Medium |
| R3 : If temperature is High | Then fan speed is Fast |

8) Inference and Defuzzification (Example Calculation) :

Given : Temperature = 35 $^\circ\text{C}$

1) Fuzzification :

$$\mu_{\text{Medium}}(35) = 0.4 \quad \mu_{\text{High}}(35) = 0.6$$

2) Rule Evaluation :

- R2 $\rightarrow$ fan = Medium (0.4)
R3 $\rightarrow$ fan = Fast (0.6)

3) Aggregation :

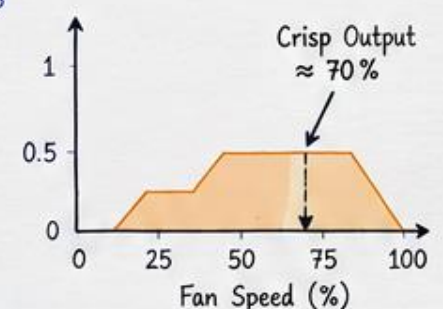
Combine outputs of all rules.

4) Defuzzification :

Using Centroid method, suppose we get:

Fan Speed = 70 % (crisp output)

Output (Aggregated Fuzzy Set)



9) Applications :

- Control systems (AC, washing machine, fan)
- Expert systems
- Decision making
- Pattern recognition
- Image processing
- Natural language processing
- AI and robotics
- Fuzzy Prolog for knowledge representation

10) Advantages & Limitations :

✓ **Advantages :**

- Handles imprecision and uncertainty
- Closer to human reasoning
- Simple and flexible
- Useful for real-world problems

✗ **Limitations :**

- No standard method for designing membership functions
- Results can be subjective
- Computationally intensive for large systems

"Fuzzy logic captures the shades of truth in the real world."



"Knowledge can be what to do or what is true."

Procedural and Declarative Knowledge

Artificial Intelligence
(7th Sem - GTU)
(Prolog)

1) Procedural Knowledge

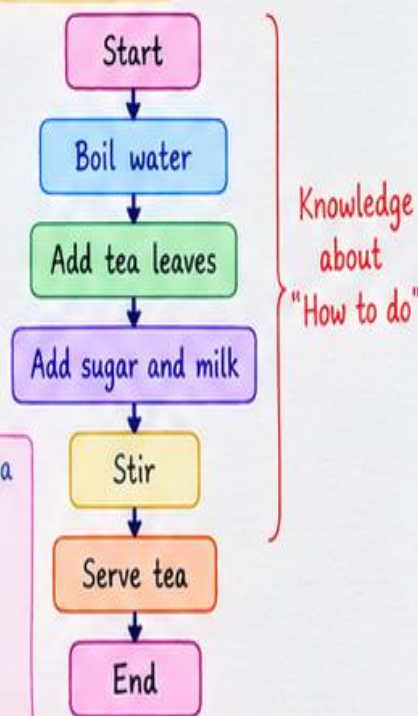
- Procedural knowledge represents "how to do something".
- It is knowledge about a sequence of actions, steps or procedures to achieve a goal.
- It specifies control, strategy and methods.
- Often represented using algorithms, rules, procedures or production rules.
- In AI, procedural knowledge is used in problem solving, planning and control (e.g., in expert systems).

2) Example : Making a Cup of Tea (Procedural)

Steps (Procedure) :

- Boil water.
- Add tea leaves.
- Add sugar.
- Add milk.
- Stir and serve.

Flowchart :



3) Representation in Prolog :

% Procedural knowledge: steps to make tea
make_tea :-

```

boil_water,
add_tea_leaves,
add_sugar,
add_milk,
stir,
serve.
  
```

```

boil_water :- write('Boiling water...'), nl.
add_tea_leaves :- write('Adding tea leaves...'), nl.
add_sugar :- write('Adding sugar...'), nl.
add_milk :- write('Adding milk...'), nl.
stir :- write('Stirring...'), nl.
serve :- write('Serving tea.').
  
```

Query : ?- make_tea.

It performs the steps in sequence.

4) Declarative Knowledge:

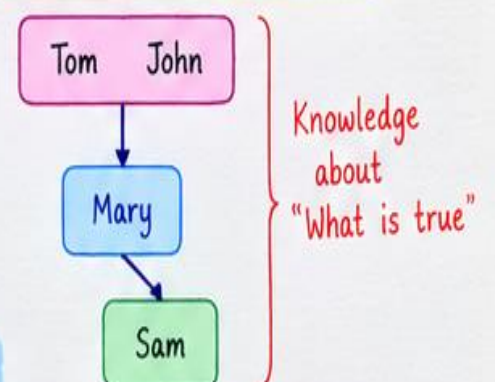
- Declarative knowledge represents "what is true" about a domain.
- It describes facts, objects, relationships and rules without specifying how to use them.
- It is concerned with the "knowledge of the world" rather than the control strategy.
- In AI, declarative knowledge is widely used in knowledge bases, logical representation, database systems etc.
- Prolog is mainly a declarative programming language.

5) Example : Family Relationship (Declarative)

Facts (What is true) :

- John is a father of Mary.
- Mary is a parent of Sam.
- Tom is a parent of Mary.

Knowledge Representation :



6) Representation in Prolog :

% Declarative knowledge: family relationships

```

parent(john, mary).
parent(mary, sam).
parent(tom, mary).
  
```

```

father(john, mary).
mother(tom, mary).
  
```

% rule (declarative)

```

grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
  
```

Query and Result :

?- grandparent(john, sam).
true.

Prolog finds the answer using the facts and rules.
(We only state what is true, not how to search.)

7) Comparison :

Aspect	Procedural Knowledge	Declarative Knowledge
Meaning	How to do something	What is true
Focus	Sequence of actions / control	Facts and relationships
Representation	Algorithms, procedures, rules	Facts and logical rules
Example	Steps to make tea	parent(john, mary).
In Prolog	Can be written, but not the main style	Mainly declarative (logic-based)
Used in	Planning, problem solving, expert systems	Knowledge bases, reasoning, query answering

"Procedural knowledge tells the agent how to act.
Declarative knowledge tells the agent what is known."

"In AI, knowledge is power, whether it tells you how or what."



"Reasoning in uncertain world."

Probabilistic Reasoning

Artificial Intelligence
(7th Sem - GTU)
(Probabilistic AI & Prolog)

1) Introduction :

- Probabilistic reasoning deals with **uncertain**, incomplete and noisy information.
- Instead of giving a definite true/false answer, it computes the **degree of belief** (probability) of a hypothesis.
- It is widely used in AI for decision making, diagnosis, expert systems, natural language processing, robotics etc.
- Mainly based on probability theory and **Bayesian inference**.

2) Key Terminology :

Term	Meaning
Random Variable	A variable that can take different possible values with certain probabilities.
Event (E)	A set of outcomes (e.g., P(Cold)).
Prior Probability	Initial probability before seeing new evidence (e.g., P(Disease)).
Posterior Probability	Updated probability after seeing evidence (e.g., P(Disease Symptom)).
Conditional Probability	$P(A B)$: probability of A given B is true.
Joint Probability	$P(A, B)$: probability of both A and B.
Marginal Probability	$P(A)$: total probability of A.

3) Types of Probabilistic Reasoning :

- Diagnostic reasoning (cause from effect)
- Predictive reasoning (effect from cause)
- Causal reasoning (reasoning with causal relations)
- Bayesian reasoning (using Bayes' theorem)
- Probabilistic inference in graphical models (e.g., Bayesian networks)

4) Example : Medical Diagnosis

A disease D occurs in 1% of the population. A test for the disease is 99% accurate if a person has the disease (true positive rate) and gives a false positive result 2% of the time for a person who does not have the disease. If a person tests positive, what is the probability that the person actually has the disease?

Given :

- $P(D) = 0.01$ (prior)
- $P(\neg D) = 0.99$
- $P(+ | D) = 0.99$ (sensitivity)
- $P(+ | \neg D) = 0.02$ (false positive rate)

Find :

$$P(D | +) = ?$$

5) Solution using Bayes' Theorem :

$$\begin{aligned}
 P(D | +) &= \frac{P(+ | D) \cdot P(D)}{P(+ | D) \cdot P(D) + P(+ | \neg D) \cdot P(\neg D)} \\
 &= \frac{(0.99)(0.01)}{(0.99)(0.01) + (0.02)(0.99)} \\
 &= \frac{0.0099}{0.0297} \approx 0.333
 \end{aligned}$$

Result :

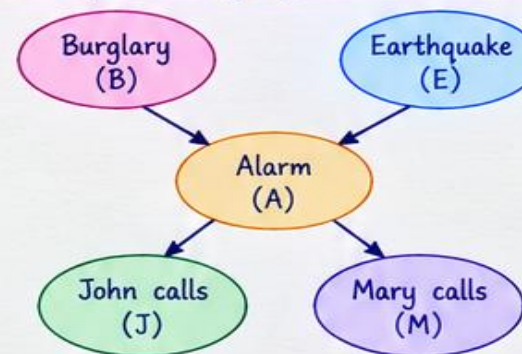
$$P(D | +) \approx 0.333 \text{ (33.3\%)}$$

Even though the test is 99% accurate, there is only 33.3% chance that the person actually has the disease!

6) Bayesian Network :

- A graphical model to represent probabilistic relationships between variables.
- Nodes represent random variables.
- Directed edges represent conditional dependencies.
- Joint probability is factorized using the network structure.

Example : Burglary Alarm Network



Joint distribution :

$$P(B, E, A, J, M) = P(B) P(E) P(A | B, E) P(J | A) P(M | A)$$

It encodes conditional dependencies and helps in probabilistic inference.

7) Inference in Bayesian Network (Example) :

Given: John calls ($J = \text{true}$).

Find: $P(\text{Burglary} | J = \text{true})$

- Use Bayes' theorem and sum over all hidden variables (A, E, M).
- In practice, exact inference can be complex, so algorithms like Variable Elimination, Belief Propagation are used.
- Probabilistic reasoning in AI systems (e.g., medical diagnosis, robotics) often uses such networks.

8) Representation in Prolog (ProbLog) :

ProbLog allows probabilistic facts and rules.

Example :

```

0.01::disease.                % P(D) = 0.01
0.99::test_positive :- disease. % P(+|D) = 0.99
0.02::test_positive :- \+ disease. % P(+|¬D) = 0.02

query(disease).                % ask probability of disease given test positive
  
```

ProbLog computes the probability of query using probabilistic inference.

9) Applications :

- Medical diagnosis systems
- Fault detection and monitoring
- Risk assessment and decision making
- Natural language processing (e.g., word sense disambiguation)
- Robotics and autonomous systems
- Expert systems

10) Advantages :

- ✓ Handles uncertainty and noisy data.
- ✓ More realistic than deterministic reasoning.
- ✓ Useful for decision making.
- ✓ Can be combined with graphical models (e.g., Bayesian networks).

11) Limitations :

- ✗ Requires prior probabilities.
- ✗ Computationally expensive for large domains.
- ✗ Exact inference can be complex (need approximate methods).
- ✗ Quality depends on correctness of probability estimates.

"Probability gives us a rational way to deal with uncertainty."



* "Knowledge is more meaningful when it is connected."

Semantic Net & Frame

Knowledge Representation using Structures

Representing Knowledge in a Structured Way

Artificial Intelligence
(7th Sem - GTU)

1) Semantic Net

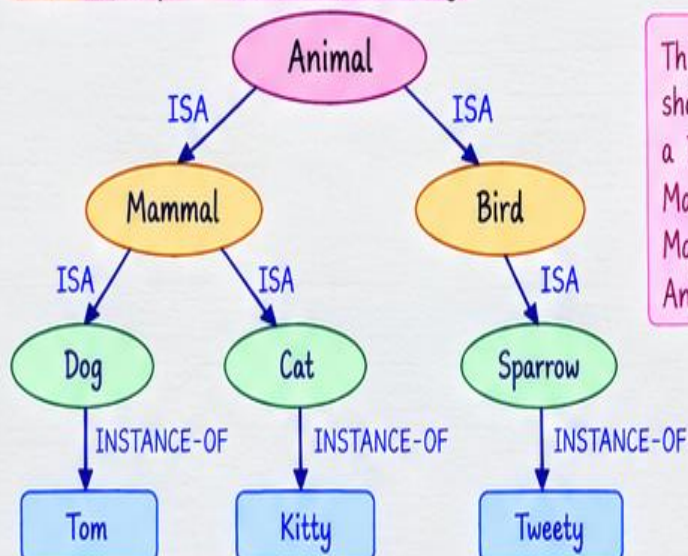
1.1) Introduction :

- Semantic Net is a graphical representation of knowledge using **nodes** and **labeled edges**.
- Nodes represent objects / concepts.
- Edges represent relationships between concepts.
- It is based on **graph theory** and is easy to understand.
- Useful for representing simple knowledge about the world (e.g., objects, categories, and relations).
- Inheritance (ISA) can be naturally represented.

1.2) Common Relations :

Relation	Meaning	Example
ISA	"is a" (class-subclass)	Dog ISA Animal
AKO	"a kind of"	Rose AKO Flower
PART-OF	Part-whole relation	Wheel PART-OF Car
HAS-A	Possession	Person HAS-A Heart
INSTANCE-OF	Instance relation	Tom INSTANCE-OF Dog

1.3) Example : Animal Hierarchy



This semantic net shows that Tom is a Dog, Dog is a Mammal and Mammal is an Animal.

1.4) Representation in Prolog :

```

isa(mammal, animal). % Mammal is an animal
isa(dog, mammal). % Dog is a mammal
isa(cat, mammal). % Cat is a mammal
isa(bird, animal). % Bird is an animal
isa(sparrow, bird). % Sparrow is a bird
instance(tom, dog). % Tom is a dog
instance(kitty, cat). % Kitty is a cat
instance(tweety, sparrow). % Tweety is a sparrow
  
```

Query Example :

```

?- isa(dog, animal).
true.
?- instance(tom, mammal).
true.
  
```

"Semantic nets provide an intuitive and visual way to represent knowledge." 😊

2) Frame

2.1) Introduction :

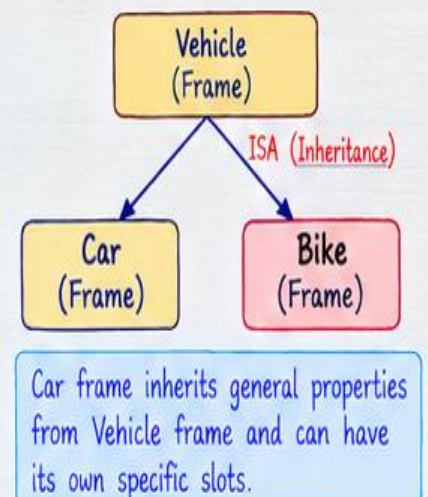
- A Frame is a structured data representation for a conceptual object.
- It consists of a collection of attributes (**slots**) and their values (slot fillers).
- It can also include **default** values, **constraints** and **procedures** (**methods**).
- Introduced by **Marvin Minsky** (1975).
- Frames are useful for representing stereotypical situations, objects, and complex structured knowledge.

2.2) Components of a Frame :

Component	Description	Example
Frame Name	Name of the object/concept	Car
Slot (Attribute)	A property of the frame	color, engine, wheels
Slot Value	Value of the attribute	red, petrol, 4
Default Value	Used if no explicit value is given	color = white
Inheritance	A frame can inherit slots from another frame	Car inherits from Vehicle
Procedural Attachment	A procedure to compute the value	start() method

2.3) Example : Frame for a Car

Frame : Car	
Slot	Value
ISA	Vehicle
Color	Red (default : White)
Number-of-Wheels	4
Engine	Petrol
Capacity	5 (default)
Start	start() (procedure)



Car frame inherits general properties from Vehicle frame and can have its own specific slots.

2.4) Representation in Prolog (using facts) :

```

frame(vehicle).
slot(vehicle, wheels, 0).
slot(vehicle, engine, unknown).
frame(car).
isa(car, vehicle).
slot(car, wheels, 4).
slot(car, engine, petrol).
slot(car, color, white). % default value
slot(car, capacity, 5).
  
```

Query Example :

```

?- isa(car, vehicle).
true.
?- slot(car, engine, X).
X = petrol.
  
```

2.5) Advantages :

- ✓ Represents complex structured knowledge.
- ✓ Supports inheritance (reduces redundancy).
- ✓ Can include default values and procedures.
- ✓ Useful in expert systems and reasoning.

2.6) Limitations :

- ✗ Lacks formal semantics (compared to logic).
- ✗ Not suitable for uncertain or probabilistic knowledge.
- ✗ Can become complex for very large domains.

"Frames help us represent the real world in a structured and modular way." 😊